

UNIDAD 2 ACTIVIDAD DE ENTREGA 1

ALEXANDER AMAYA ARANGO

POLITECNICO GRANCOLOMBIANO

ARQUITECTURA DE APLICACIONES WEB

ING. WILSON EDUARDO SOTO FORERO

MODALIDAD DE FORMACIÓN VIRTUAL

25 de agosto de 2026

TABLA DE CONTENIDO

1. INFORMACIÓN GENERAL	5
2. INTRODUCCIÓN.....	5
3. ARQUITECTURA DE LA SOLUCIÓN.....	5
3.1 Diagrama de Arquitectura por Capas.....	5
3.2 Estructura de Paquetes.....	6
3.3 Principios SOLID Aplicados	7
4. TECNOLOGÍAS UTILIZADAS.....	7
5. ENDPOINTS DE LA API	7
5.1 Ejemplo de Petición y Respuesta.....	8
5.2 Ejemplo de Respuesta de Error	8
5.3 Evidencia de Pruebas.....	9
5.3.1 POST - Crear producto	9
5.3.2 GET - Listar todos	10
5.3.3 GET - Obtener por ID.....	10
5.3.4 PUT – Actualizar	11
5.3.5 DELETE – Eliminar	11
6. URL VIDEO EXPLICATIVO	12
7. REPOSITORIO GITHUB:	12
8. CONCLUSIONES.....	12
9. BIBLIOGRAFIA	13

TABLA DE ILUSTRACIONES

Ilustración 1. Diagrama de Arquitectura por Capas	5
Ilustración 2. Diagrama de Paquetes.	6
Ilustración 3. Ejemplo de Petición y Respuesta	8
Ilustración 4. Ejemplo de Respuesta de Error	8
Ilustración 5. Solicitud POST - Crear producto.....	9
Ilustración 6. Solicitud GET - Listar todos.....	10
Ilustración 7. Solicitud GET - Obtener por ID.	10
Ilustración 8. Solicitud PUT – Actualizar.....	11
Ilustración 9. Solicitud DELETE – Eliminar.....	11
Ilustración 10. Solicitud GET - Obtener por ID, después de eliminar el producto	12

INDICE DE TABLAS

Tabla 1. Aplicación de los Principios SOLID en la Arquitectura en Capas.	7
Tabla 2. Stack Tecnológico y Herramientas de Desarrollo.....	7
Tabla 3. Especificación de Endpoints y Operaciones de la API REST.	7

1. INFORMACIÓN GENERAL

Módulo: Arquitectura de Aplicaciones Web
Actividad: Unidad 2 Actividad de Entrega 1
Tema: Backend con servicios RESTful CRUD
Estudiante: Alexander Amaya Arango

2. INTRODUCCIÓN

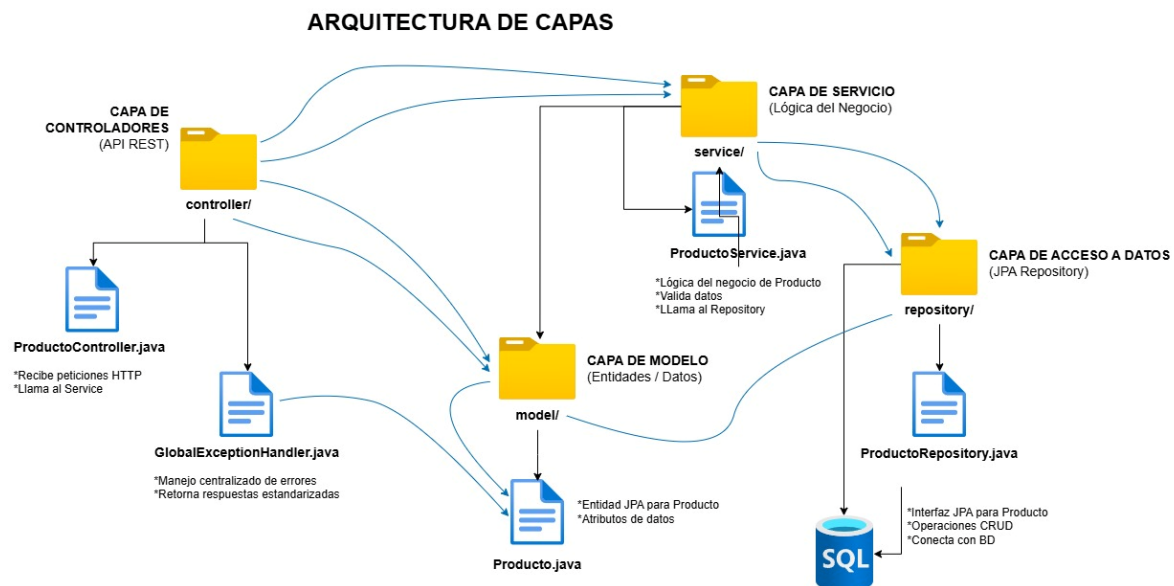
El documento adjunto tiene como objetivo complementar la entrega de la actividad sumativa de la Unidad 2 del módulo de Arquitectura de Aplicaciones Web. En este trabajo se desarrolló el backend de una aplicación web que implementa servicios API REST para operaciones CRUD (Create, Read, Update, Delete) sobre una base de datos.

El proyecto fue realizado con el framework Spring Boot, Spring Data JPA como ORM y el motor de base de datos en memoria H2 Database. Se ha seguido el patrón de arquitectura por capas en la implementación de la arquitectura, lo cual garantiza una clara separación de responsabilidades y facilita el mantenimiento y escalabilidad del código.

3. ARQUITECTURA DE LA SOLUCIÓN

3.1 Diagrama de Arquitectura por Capas

Ilustración 1. Diagrama de Arquitectura por Capas

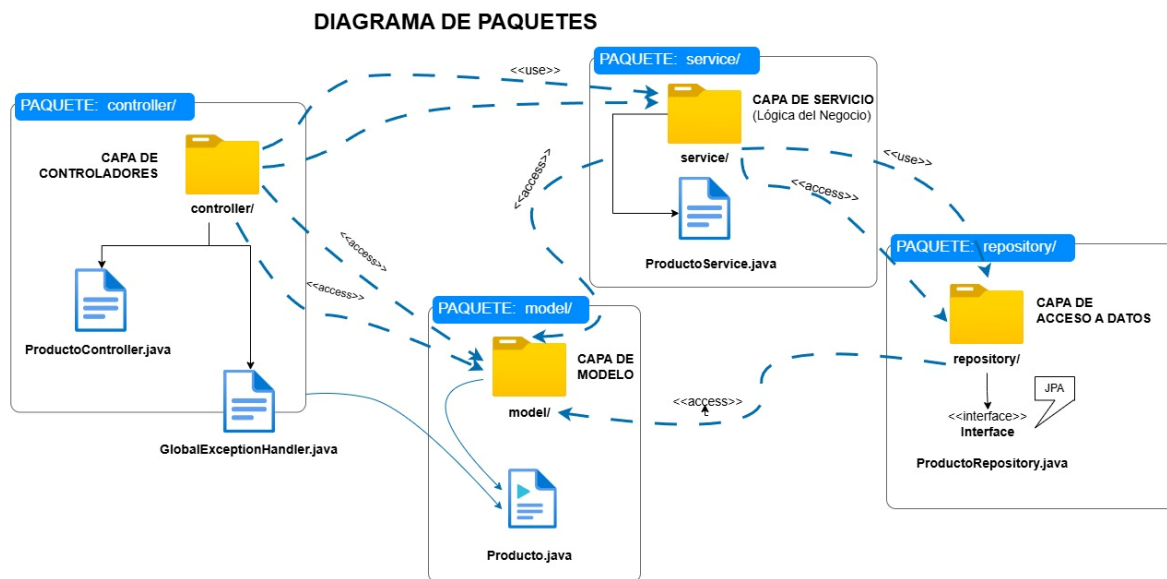


Nota: La arquitectura sigue el patrón por capas: controller expone la API REST, service contiene la lógica de negocio, repository gestiona el acceso a datos con JPA, y model define la entidad Producto.

El `GlobalExceptionHandler` centraliza el manejo de errores, asegurando respuestas consistentes. Esta organización promueve mantenibilidad y escalabilidad. Elaboración propia.

3.2 Estructura de Paquetes

Ilustración 2. Diagrama de Paquetes.



Nota: El diagrama de paquetes por capas: controller maneja las peticiones HTTP, service contiene la lógica de negocio, repository gestiona el acceso a datos con JPA, y model define la entidad Producto. El `GlobalExceptionHandler` centraliza el manejo de errores, asegurando respuestas consistentes. Elaboración propia.

3.3 Principios SOLID Aplicados

Tabla 1. Aplicación de los Principios SOLID en la Arquitectura en Capas.

Principio	Implementación
S - Responsabilidad Única	Cada clase tiene una responsabilidad única: Controller maneja HTTP, Service contiene lógica de negocio, Repository accede a datos
O - Abierto/Cerrado	Se pueden agregar nuevos endpoints o servicios sin modificar los existentes
L - Sustitución de Liskov	Las dependencias se inyectan mediante interfaces (JpaRepository)
I - Segregación de Interfaces	JpaRepository proporciona métodos específicos sin forzar implementaciones innecesarias
D - Inversión de Dependencias	Los módulos de alto nivel (Service) no dependen de módulos de bajo nivel (Repository), ambos dependen de abstracciones

Nota: Cumplimiento de los principios de diseño de software SOLID dentro de la estructura y distribución de responsabilidades del sistema. Elaboración propia.

4. TECNOLOGÍAS UTILIZADAS

Tabla 2. Stack Tecnológico y Herramientas de Desarrollo.

Tecnología	Versión	Propósito
Spring Boot	3.2.x	Framework principal para el backend
Spring Data JPA	3.2.x	ORM para acceso a datos (Hibernate)
H2 Database	2.2.x	Base de datos en memoria para desarrollo
Lombok	1.18.x	Reducción de código boilerplate
Maven	3.9.x	Gestión de dependencias y construcción
Java	21 (LTS)	Lenguaje de programación

Nota: Conjunto de tecnologías, dependencias y herramientas utilizadas para el desarrollo de la aplicación backend. Elaboración propia.

5. ENDPOINTS DE LA API

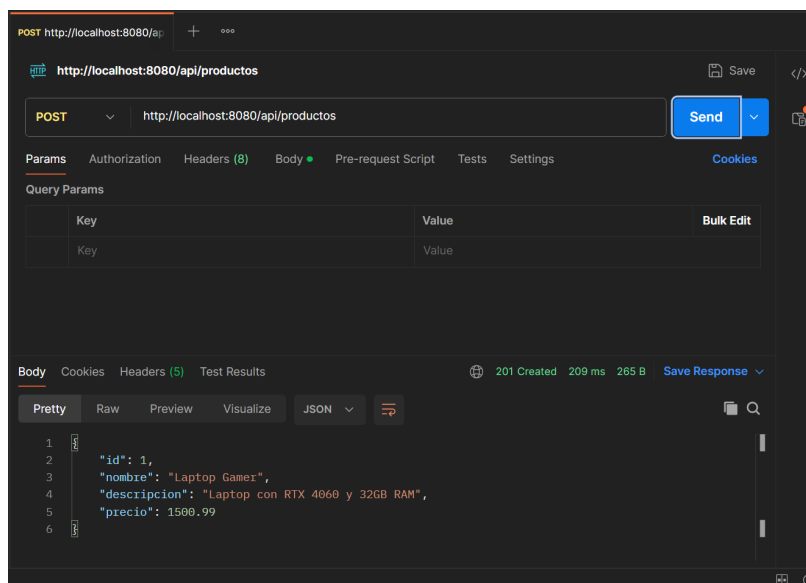
Tabla 3. Especificación de Endpoints y Operaciones de la API REST.

Método HTTP	Endpoint	Descripción	Código de Éxito	Código de Error
POST	/api/productos	Crear un nuevo producto	201 Created	400 Bad Request
GET	/api/productos	Obtener todos los productos	200 OK	500 Internal Server Error
GET	/api/productos/{id}	Obtener un producto por ID	200 OK	404 Not Found
PUT	/api/productos/{id}	Actualizar un producto	200 OK	404 Not Found
DELETE	/api/productos/{id}	Eliminar un producto	204 No Content	404 Not Found

Nota: Especificación de los *endpoints* de la API REST, los métodos HTTP utilizados, las operaciones CRUD asociadas y los códigos de respuesta esperados para casos de éxito y error. Elaboración propia.

5.1 Ejemplo de Petición y Respuesta

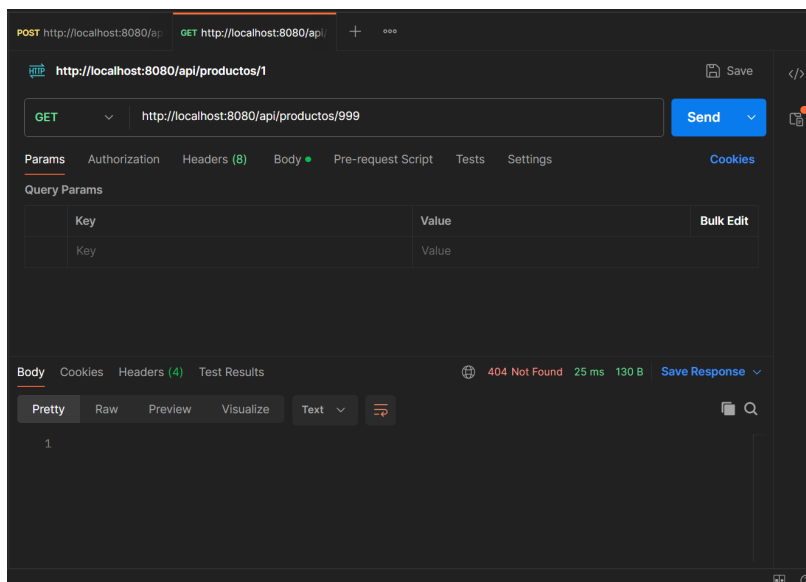
Ilustración 3. Ejemplo de Petición y Respuesta



Nota: Creación exitosa de un recurso en el endpoint `/api/productos` mediante una solicitud de tipo POST. Se observa la respuesta del servidor en formato JSON con la confirmación de los datos generados (`id: 1`) y el código de estado HTTP 201 Created. Elaboración propia.

5.2 Ejemplo de Respuesta de Error

Ilustración 4. Ejemplo de Respuesta de Error

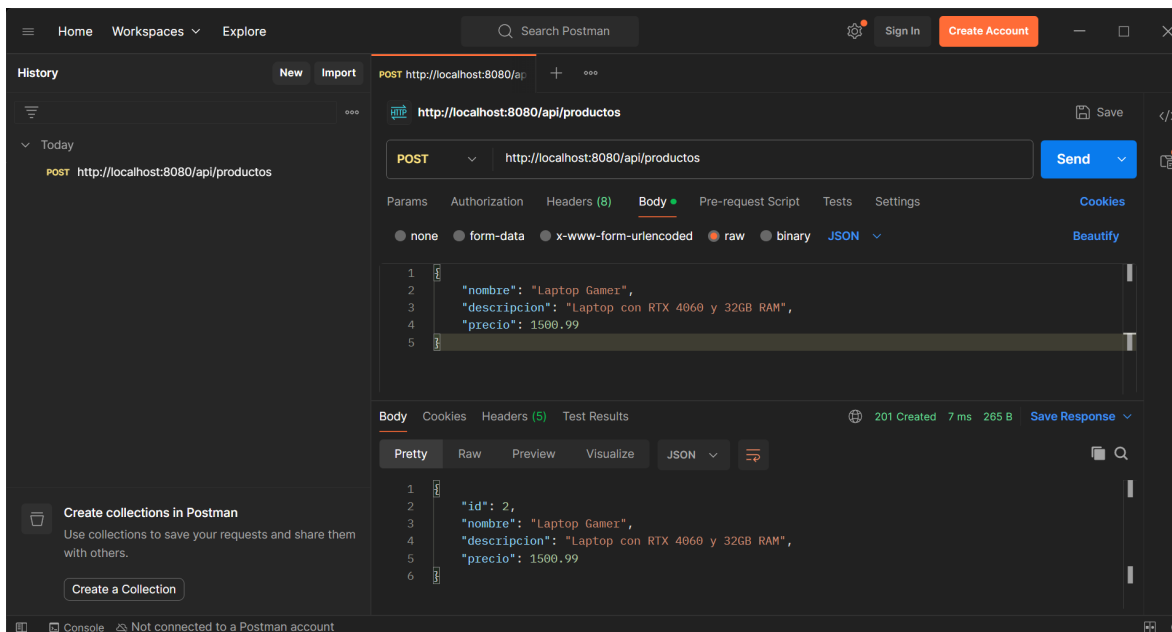


Nota: Respuesta de error que muestra una solicitud GET realizada al endpoint `/api/productos/999`. Dado que el identificador especificado no se encuentra registrado en el sistema, el servidor retorna correctamente el código de estado HTTP 404 Not Found. Elaboración propia.

5.3 Evidencia de Pruebas

5.3.1 POST - Crear producto

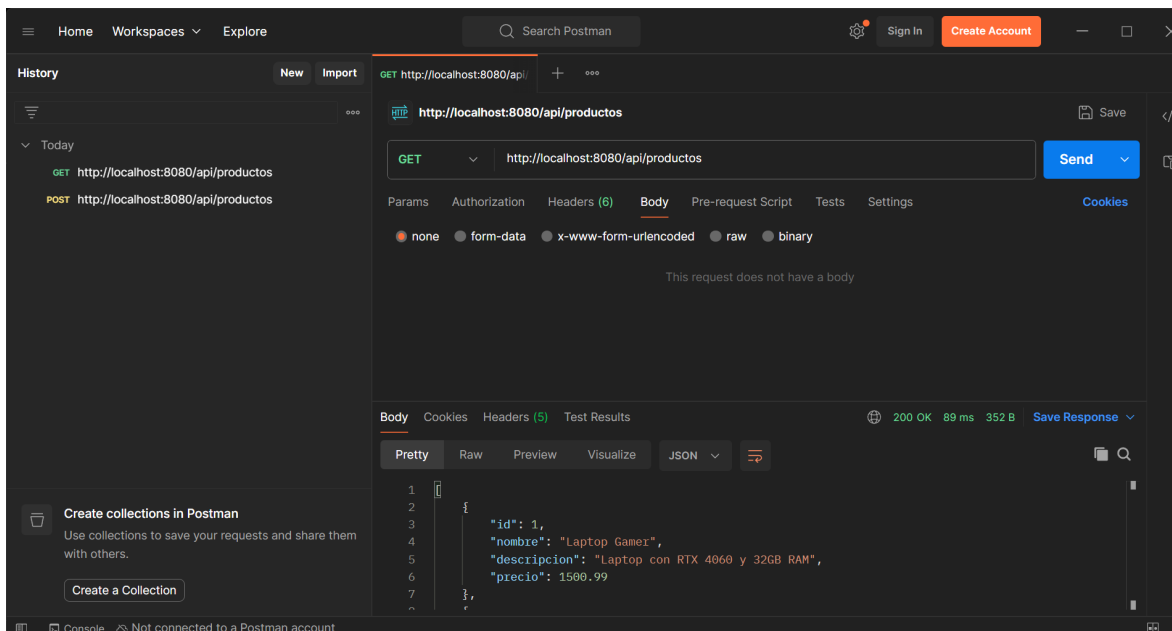
Ilustración 5. Solicitud POST - Crear producto.



Nota: Solicitud POST exitosa al *endpoint* `http://localhost:8080/api/productos`. Se aprecia el cuerpo de la solicitud (*Body*) en formato JSON con los datos del producto a crear. Como respuesta se observa la respuesta del servidor con el código de estado 201 Created y el cuerpo de la respuesta que confirma la creación del recurso e incluye el identificador único asignado ("id": 2). Elaboración propia.

5.3.2 GET - Listar todos

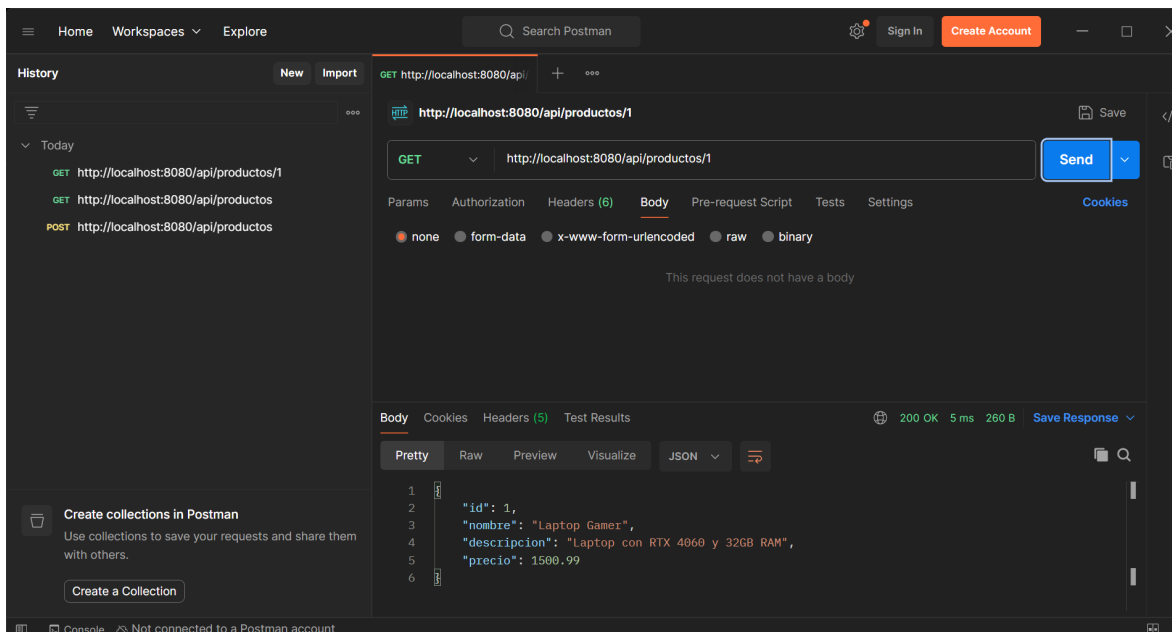
Ilustración 6. Solicitud GET - Listar todos.



Nota: Solicitud GET enviada al *endpoint* `/api/productos`. El servidor devuelve el código de estado `200 OK` junto con la lista completa de productos registrados en formato JSON. Elaboración propia.

5.3.3 GET - Obtener por ID

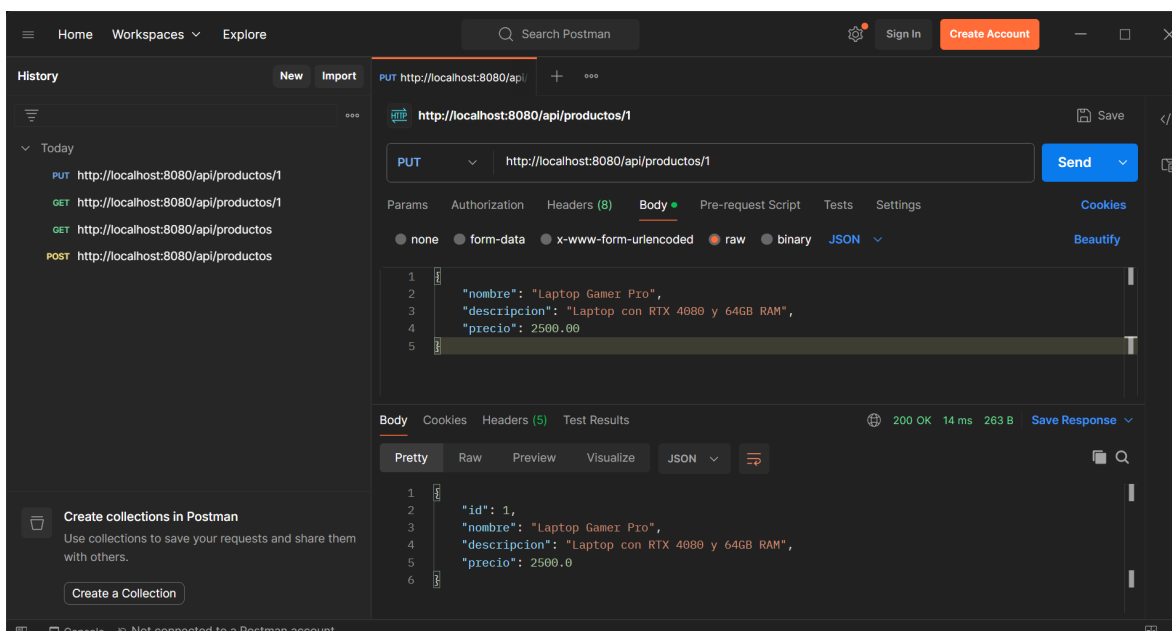
Ilustración 7. Solicitud GET - Obtener por ID.



Nota: Solicitud GET al *endpoint* `/api/productos/1`. El servidor retorna el código de estado `200 OK` junto con los detalles del producto correspondiente al identificador solicitado en formato JSON. Elaboración propia.

5.3.4 PUT – Actualizar

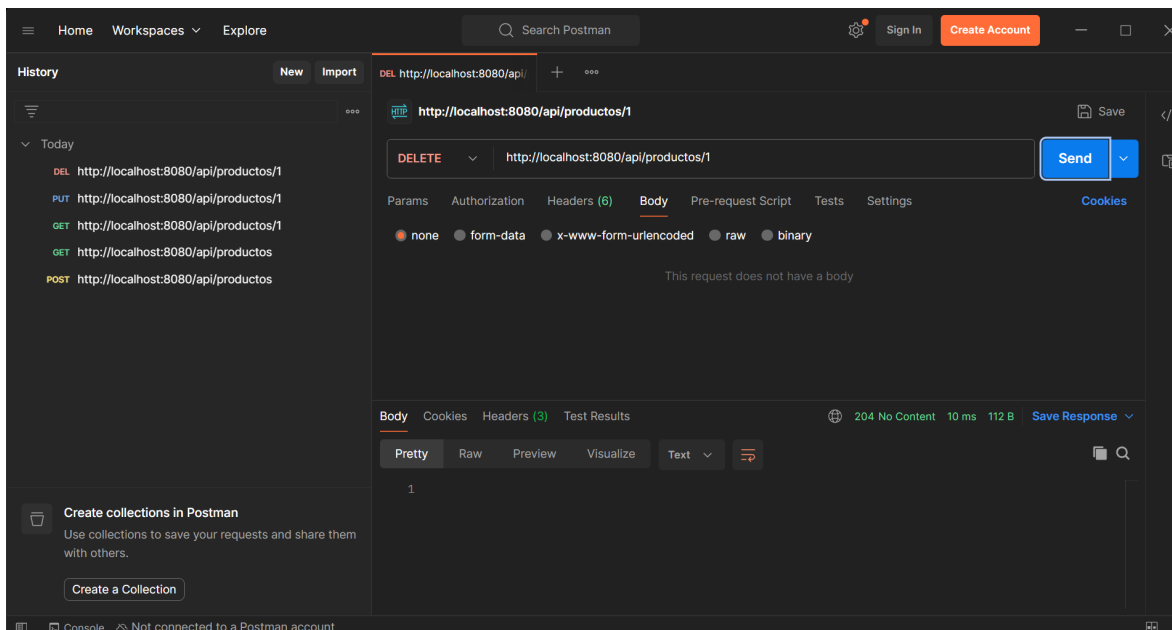
Ilustración 8. Solicitud PUT – Actualizar



Nota: Solicitud PUT enviada al *endpoint* `/api/productos/1` con los datos actualizados del producto en formato JSON. El servidor responde con el código de estado 200 OK, devolviendo el objeto modificado y confirmando la actualización en la base de datos. Elaboración propia.

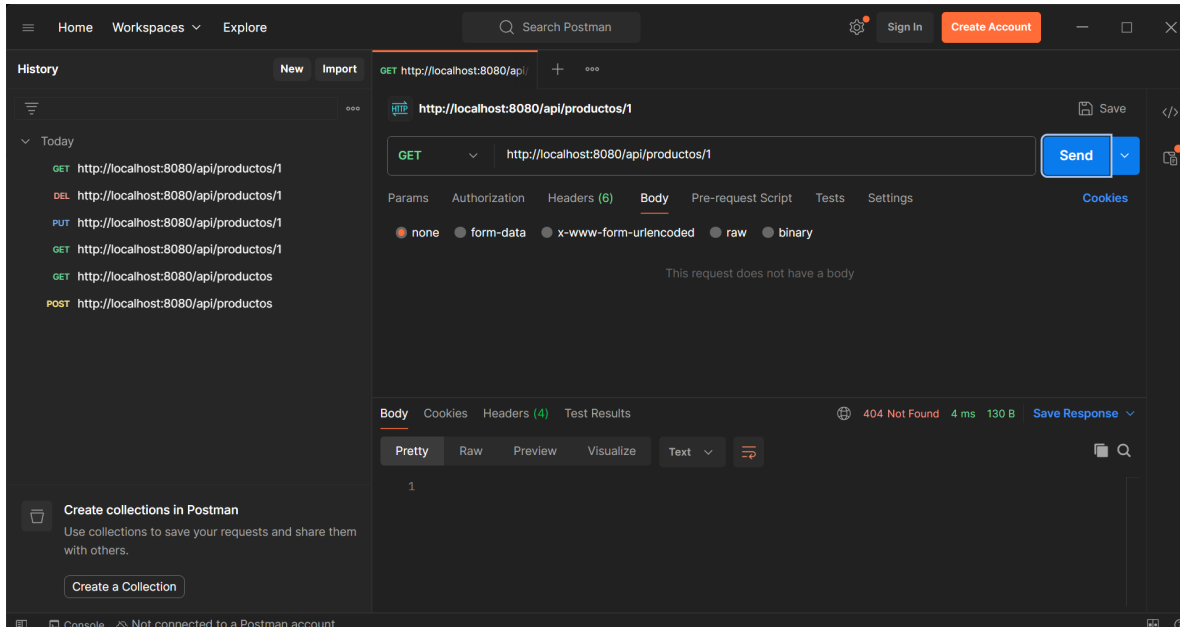
5.3.5 DELETE – Eliminar

Ilustración 9. Solicitud DELETE – Eliminar.



Nota: Solicitud DELETE enviada al *endpoint* /api/productos/1. El servidor retorna el código de estado 204 No Content, confirmando la eliminación del producto de forma exitosa y sin devolver cuerpo en la respuesta. Elaboración propia.

Ilustración 10. Solicitud GET - Obtener por ID, después de eliminar el producto



Nota: Solicitud GET al *endpoint* /api/productos/1 realizada tras la eliminación del registro. La API responde con el código de estado 404 Not Found, confirmando la remoción exitosa del producto en la base de datos y la correcta validación de recursos no encontrados. Elaboración propia.

6. URL VIDEO EXPLICATIVO

En la siguiente URL de YouTube se encuentra alojado el video del proyecto Backend-CRUD:

<https://youtu.be/d8988J3z0S8>

7. REPOSITORIO GITHUB:

En la siguiente URL de GitHub se encuentra alojado el proyecto Backend-CRUD:

<https://github.com/amayaagro/backend-crud.git>

8. CONCLUSIONES

Creación de una arquitectura sólida y escalable: Se desarrolló con éxito un servicio API RESTful backend utilizando el framework Spring Boot, organizando el código siguiendo un patrón de arquitectura por capas (controller, service, repository y model). Esta

estructura permite que haya una clara separación de responsabilidades, lo que facilita el mantenimiento, la modularidad y la escalabilidad del sistema en el futuro.

Cumplimiento de los Principios SOLID: La aplicación de los principios SOLID dio fuerza a la calidad técnica del proyecto. El uso de interfaces como JpaRepository para la inversión de dependencias y la separación de lógica de negocio respecto al manejo de peticiones HTTP, asegura un código desacoplado y adaptable a nuevos requerimientos sin afectar la estabilidad del sistema.

Persistencia y Gestión eficaz de Datos: Spring Data JPA integrada con H2 Database facilitó el manejo de operaciones CRUD en la base de datos. Además, el manejo centralizado de excepciones a través de GlobalExceptionHandler permitió estandarizar las respuestas de la API ante posibles errores, retornando códigos de estado HTTP coherentes y profesionales (201 Created, 204 No Content y 404 Not Found).

Práctica de validación y cumplimiento de los estándares REST: Las pruebas realizadas en Postman permitieron comprobar que cada una de las peticiones HTTP implementadas se ejecutan y persisten correctamente. Se probó que la API cumple con el estándar REST, gestionando correctamente los recursos mediante sus URIs y los códigos de respuesta correspondientes.

Optimización a través del Ciclo de Desarrollo: La construcción del software se ha realizado de forma ágil gracias a la utilización del ecosistema Spring y dependencias como Lombok y Maven. Los esfuerzos se pudieron concentrar en la lógica de negocio y en la consistencia de la arquitectura, gracias a que se redujo el código repetitivo (boilerplate) y se gestionaron automáticamente las bibliotecas.

Aislamiento y eficiencia en los entornos de prueba: Incorporando el motor de base de datos H2 en memoria se obtuvo un entorno liviano, rápido e independiente para validar las operaciones CRUD. Es una configuración muy adecuada para etapas de desarrollo y pruebas continuas, ya que no requiere dependencias de infraestructura externa, ni de configuraciones complejas de motores relacionales pesados.

Estándar de Gestión de Errores y Calidad de Respuesta: Se implementó una estrategia global de control de excepciones, lo que permitió que el sistema devuelva estructuras de datos homogéneas e informativas. Esto hace que la experiencia de consumo del servicio desde el cliente (frontend o aplicaciones de prueba como Postman) mejore significativamente, facilitando la identificación y corrección de fallos en tiempo de ejecución.

9. BIBLIOGRAFIA

Spring Boot Reference Guide [en línea]. Spring Boot Documentation.

<https://docs.spring.io/spring-boot/docs/current/reference/html/>

Spring Data JPA Reference Documentation [en línea]. Spring Data JPA.
<https://docs.spring.io/spring-data/jpa/reference/>

H2 Database Engine Documentation [en línea]. H2 Database.
<https://www.h2database.com/html/main.html>

Project Lombok Features [en línea]. Project Lombok. <https://projectlombok.org/features/>

Apache Maven Project Documentation [en línea]. Apache Maven.
<https://maven.apache.org/guides/>

Java Platform, Standard Edition Documentation. Oracle Corporation.
<https://docs.oracle.com/javase/specs/>

NeverSight (2024). Principios SOLID en Arquitectura de Software. GitHub.
https://github.com/NeverSight/skills_feed/blob/main/data/skills-md/bbeierle12/skill-mcp-claude/software-architecture/SKILL.md

Vogella, L. (2023). REST with Java (JAX-RS) using Jersey.
<https://www.vogella.com/tutorials/REST/article.html>

Pluralsight (2024). Spring Certified Professional - Developing REST Applications with Spring MVC. <https://www.pluralsight.com/labs/>

Pressman, R. S. (2019). Ingeniería del Software: Un Enfoque Práctico (4ª ed.). México: McGraw-Hill. ISBN: 978-1-4567-8969-1.
<https://www.calameo.com/books/000409775b2733e206ae1>

Sommerville, I. (2018). Ingeniería de Software (10ª ed.). Madrid: Pearson Educación. ISBN: 978-1-292-09812-6. <https://archive.org/details/sommerville-engenharia-de-software-10e>

Fowler, M. (2017). Patterns of Enterprise Application Architecture. Boston: Addison-Wesley. ISBN: 978-0-321-12742-6. <https://desol-tech.com/buckets/Patterns%20of%20Enterprise%20Application%20Architecture.pdf>